

Nama :adityah candra Pratama

Kelas 2025 B

Nim 25091397046

## **A. Analisis & Desain Algoritma Sorting Lanjutan**

### **a. Ruang & Distribusi Sort**

Implementasi standar Radix Sort menggunakan array berisi 10 queue membutuhkan memori tambahan karena setiap digit harus dipindahkan sementara ke bucket/queue. Pada Modul A dengan data  $10^6$  elemen dan batas  $O(1)$  extra space, pendekatan ini tidak memenuhi syarat karena queue dapat menyimpan banyak elemen tambahan sehingga kompleksitas ruang menjadi  $O(n)$ .

Pada Merge Sort versi improved, hanya digunakan satu tmpArray global berukuran  $n$  sejak awal. Berbeda dengan versi slice yang membuat sub-array baru di setiap rekursi, pendekatan virtual sublists hanya memakai indeks first, mid, dan last untuk menandai bagian array. Dengan begitu overhead memori jauh lebih kecil karena tidak ada alokasi array berulang.

- kompleksitas waktu tetap mendekati  $O(n \log n)$
- Namun jumlah swap meningkat
- Cache locality lebih buruk
- Stabilitas sering hilang

### **b. Linked List & Pointer Manipulation**

Pada linked list, Merge Sort lebih unggul karena tidak membutuhkan akses acak seperti array.

Fungsi `_splitLinkedList()` menggunakan teknik fast-slow pointer:

- midPoint bergerak 1 langkah
- curNode bergerak 2 langkah

Ketika curNode mencapai akhir list, midPoint otomatis berada di tengah. Teknik ini menemukan titik tengah hanya dengan satu traversal tanpa perlu menghitung panjang list terlebih dahulu.

Proses merge memakai dummy node dan tail reference:

- dummy menjadi kepala sementara
- tail menunjuk node terakhir hasil merge
- Node lama langsung disambungkan ulang menggunakan `.next`

Karena hanya memodifikasi pointer tanpa membuat node baru, tidak ada alokasi memori tambahan. Kompleksitas ruang akhirnya hanya  $O(\log n)$  akibat stack rekursi.

### **c. Quick Sort Worst-Case & Pivot Strategy**

Pada Quick Sort, jika data descending dan pivot selalu dipilih dari elemen pertama:

- Semua elemen berada di satu sisi pivot
- Pointer left dan right menghasilkan partisi sangat tidak seimbang
- Rekursi menjadi sepanjang  $n$  level

Akibatnya:

- Kedalaman rekursi =  $O(n)$
- Kompleksitas waktu naik menjadi  $O(n^2)$

Strategi pivot yang lebih robust:

1. Median-of-three
2. Random pivot
3. Median-of-medians

Strategi paling realistis adalah median-of-three karena ringan dan cukup efektif menghindari worst-case.

Namun pada singly linked list, strategi ini kurang efisien karena:

- Tidak ada akses indeks langsung
- Mengambil middle node membutuhkan traversal  $O(n)$

Akibatnya biaya pemilihan pivot meningkat dan Quick Sort menjadi kurang cocok untuk linked list dibanding Merge Sort.

#### **d. Batas Teoretis & Paradigma Sorting**

Batas bawah  $\Omega(n \log n)$  hanya berlaku pada comparison sort, yaitu algoritma yang bekerja murni dengan operasi perbandingan ( $<$ ,  $>$ ,  $<=$ ).

Radix Sort tidak melanggar teori tersebut karena:

- Tidak membandingkan elemen secara langsung
- Mengelompokkan berdasarkan digit

Dua asumsi penting:

1. Banyak digit  $d$  relatif kecil/konstan
2. Rentang nilai digit  $k$  terbatas

Karena itu kompleksitas bisa menjadi  $O(dn)$ , mendekati linear.

Jika  $d$  atau  $k$  sangat besar:

- Memori bucket meningkat drastis
- Waktu distribusi digit ikut membesar
- Keunggulan Radix Sort hilang

## B. TEORI & ANALISIS ALGORITMA BINARY TREE

### a. Pohon Ekspresi & Traversal

Ekspresi:

$$((8 * 5) + (9 / (7 - 4)))$$

dibangun menjadi pohon:

- + sebagai root
- kiri:  $8 * 5$
- kanan:  $9 / (7 - 4)$

Traversal postorder menghasilkan postfix otomatis karena:

1. subtree kiri diproses
2. subtree kanan diproses
3. operator diproses terakhir

Sehingga urutan menjadi:

$$8\ 5\ *\ 9\ 7\ 4\ -\ /\ +$$

Traversal inorder membutuhkan tanda kurung eksplisit agar prioritas operasi tidak berubah.

Jika tinggi pohon =  $h$ , maka maksimum stack rekursi `_buildString()` adalah  $O(h)$ .

### b. Struktur Heap & Pemetaan Array

Rumus:

- $\text{parent} = (i - 1) // 2$
- $\text{left} = 2 * i + 1$

hanya valid pada complete binary tree karena node harus tersusun rapat tanpa celah.

Pada `sift-down()`:

1. root dibandingkan dengan anak kiri dan kanan
2. ditukar dengan anak terbesar
3. proses diulang sampai heap property pulih

Jumlah maksimum perbandingan:

- Tinggi heap  $\approx \log_2(n)$
- Tiap level maksimal 2 perbandingan

Maka total maksimum:

$$2 \log_2(n)$$

### c. Heapsort In-Place vs Simple

Perbedaan:

| Aspek           | Simple HeapSort | In-Place HeapSort |
|-----------------|-----------------|-------------------|
| Ruang Tambahan  | $O(n)$          | $O(1)$            |
| Cache Locality  | Lebih buruk     | Lebih baik        |
| Risiko Overflow | Lebih tinggi    | Lebih rendah      |

In-place heapsort tetap  $O(n \log n)$  karena:

- Build heap =  $O(n)$
- Tiap ekstraksi =  $O(\log n)$
- Dilakukan  $n$  kali

Total:

$O(n \log n)$

### d. Decision Tree & Lower Bound

Heap Sort tidak melanggar  $\Omega(n \log n)$  karena tetap memakai perbandingan antar elemen.

Pada decision tree seperti decoding Morse:

- null link menunjukkan token tidak valid
- jika traversal menuju node kosong, input salah

Pendekatan rekursif lebih cocok karena:

- Struktur pohon bersifat hierarkis
- Rekursi alami mengikuti cabang kiri/kanan
- Kode lebih sederhana dan mudah dipahami